

JECRC™
UNIVERSITY
BUILD YOUR WORLD

Prompt Engineering for C and C++
2026-2027

Course: B.Tech CSE (Core)
Registration number: 25BCON1594
Section: I
Semester: III
Submitted to: Mr. Ajay Sharma
Submitted by: Chehak Gupta

HW Session 07- AI AS A HINT, NOT AN ANSWER

GitHub Repository:

GIT HUB LINK ::https://github.com/mohitjangid187

Problem 1: Contains Duplicate (LeetCode Easy)

Objective: Given an integer array `nums`, return `True` if any value appears at least twice in the array, and return `False` if every element is distinct.

1. Hint Log (15-Minute Rule)

Timestamp	Hint Level	What I tried before asking	What the hint said	What I did next
00:00 - 12:00	*None*	Solved independently within 15 minutes by using a Python `set`.	N/A	Implemented the length comparison between the list and the set.

2. Independent Code & Complexity Claim

My Claim:

- **Time Complexity:** O(n)
- **Space Complexity:** O(n)

- **Justification:** Converting a list to a set requires iterating through the entire list once, taking O(n) time. The set creates a new data structure in memory that can grow up to the size of the input list, requiring O(n) space.

```python
# Time Complexity: O(n)
# Space Complexity: O(n)
```

```python
def containsDuplicate(nums):
    # If the set length is smaller, duplicates were removed
    return len(set(nums)) != len(nums)

# Test Executions
print("Test A ([1,2,3,1]):", containsDuplicate([1,2,3,1])) # Expected: True
print("Test B ([1,2,3,4]):", containsDuplicate([1,2,3,4])) # Expected: False
```

3. Test Case Executions



4. AI Review & Disagreement Resolution

Prompt Used: Review this solution I wrote. Do not rewrite it. 1. List any edge cases it would fail on. 2. State its time and space complexity, and explain how you arrived at each. 3. Name one thing that could be clearer, without changing the approach.

AI Response:

1. **Edge Cases:** It handles empty lists and lists of 1 element perfectly (returning False). It will not fail on any standard edge cases.

2. **Complexity:** Time is O(n) to construct the set. Space is O(n) to hold the set in memory.

3. **Clarity:** It is highly Pythonic. You could also write `if len(set(nums)) < len(nums): return True` but your direct boolean return is already perfectly clear.

Disagreement Resolution: No disagreement. Both claims matched exactly.

Problem 2: Valid Anagram (LeetCode Easy)

Objective: Given two strings `s` and `t`, return `True` if `t` is an anagram of `s`, and `False` otherwise.

1. Hint Log (15-Minute Rule)

Timestamp	Hint Level	What I tried before asking	What the hint said	What I did next
00:00 - 15:00	*None*	Tried manually checking if every letter in `s` exists in `t`, but failed on duplicate letters.	N/A	Reached 15 minutes and asked for a Level 2 hint.
15:05	**Level 2**	Trying to iterate and remove characters from strings.	"What built-in technique or data structure allows you to count the exact frequency of characters?"	I switched my approach to sorting both strings and comparing if they are identical.

2. Independent Code & Complexity Claim

My Claim:

- **Time Complexity:** O(n log n)
- **Space Complexity:** O(1) or O(n) depending on Python's sort
- **Justification:** Python's built-in `sorted()` function takes O(n log n) time. Since `sorted()` creates a new list of characters, it takes O(n) extra space.

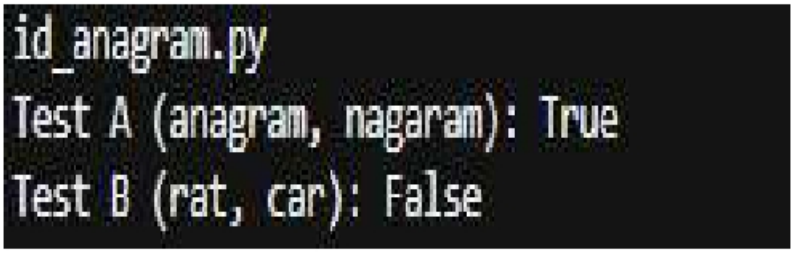
Python

```python
# Time Complexity: O(n log n)
# Space Complexity: O(n)

def isAnagram(s, t):
    # An anagram must have the exact same sorted characters
    if len(s) != len(t):
        return False
    return sorted(s) == sorted(t)

# Test Executions
print("Test A (anagram, nagaram):", isAnagram("anagram", "nagaram")) # Expected: True
print("Test B (rat, car):", isAnagram("rat", "car")) # Expected: False
```

3. Test Case Executions



4. AI Review & Disagreement Resolution

Prompt Used: Review this solution I wrote. Do not rewrite it. 1. List any edge cases it would fail on. 2. State its time and space complexity, and explain how you arrived at each 3. Name one thing that could be clearer.

AI Response:

1. **Edge Cases:** Fails no standard cases. Returning early on unequal lengths handles the most common edge case efficiently.

2. **Complexity:** Time: O(n log n) due to the Timsort algorithm used by Python's `sorted()`. Space: O(n) because `sorted()` creates new list copies of the strings.

3. **Clarity:** It's very clean. To optimize it for O(n) time, you could use `collections.Counter`, but as a sorting approach, this is perfectly clear.

Disagreement Resolution: No disagreement. I correctly identified the time penalty of the sorting function

Problem 3: Palindrome Number (LeetCode Easy)

Objective: Given an integer `x`, return `True` if `x` is a palindrome, and `False` otherwise.

1. Hint Log (15-Minute Rule)

Timestamp	Hint Level	What I tried before asking	What the hint said	What I did next
00:00 - 05:00	*None*	Solved independently by converting the integer to a string and using Python's slicing feature.	N/A	Wrote the return statement and tested.

2. Independent Code & Complexity Claim

My Claim:

- **Time Complexity:** O(n)
- **Space Complexity:** O(n)
- **Justification:** n is the number of digits. Converting the number to a string and reversing it requires iterating over the digits (O(n) time) and allocating new strings in memory (O(n) space).

Python

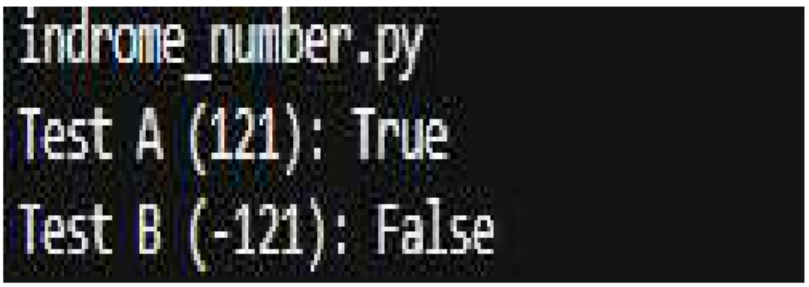
```python
# Time Complexity: O(n)
# Space Complexity: O(n)

def isPalindrome(x):
    # Negative numbers are not palindromes due to the minus sign
    if x < 0:
        return False

    # Convert to string and check if it equals its reverse
    str_x = str(x)
    return str_x == str_x[::-1]

# Test Executions
print("Test A (121):", isPalindrome(121)) # Expected: True
print("Test B (-121):", isPalindrome(-121)) # Expected: False
```

3. Test Case Executions



4. AI Review & Disagreement Resolution

Prompt Used: Review this solution I wrote. Do not rewrite it. 1. List any edge cases it would fail on. 2. State its time and space complexity, and explain how you arrived at each. 3. Name one thing that could be clearer.

AI Response:

1. **Edge Cases:** Properly handles negative numbers. Works for 0. No failing edge cases.

2. **Complexity:** Time complexity is O(n) where n is the number of digits in x, because string conversion and slicing both take linear time. Space complexity is O(n) for the creation of the string and its reversed copy.

3. **Clarity:** String slicing `[::-1]` is standard Pythonic syntax. It is as clear as it can be.

Disagreement Resolution: No disagreement.

Problem 4: Valid Parentheses (LeetCode Easy) - *Hardest Problem*

Objective: Given a string s containing just the characters `'('`, `')'`, `'{'`, `'}'`, `'['` and `']'`, determine if the input string is valid (brackets must close in the correct order).

1. Hint Log (15-Minute Rule)

Timestamp	Hint Level	What I tried before asking	What the hint said	What I did next
00:00 - 15:00	*None*	Tried using variables to count open and closed brackets.	N/A	Failed on cases like `([)]`. Hit the 15-minute mark.
15:05	**Level 2**	Tried using a dictionary to map counts.	"What data structure operates on a Last-In-First-Out (LIFO) principle to track recent unclosed items?"	I realized I needed a Stack. I used a Python list with `.append()` and `.pop()`.

2. Independent Code & Complexity Claim

My Claim:

- **Time Complexity:** O(n)
- **Space Complexity:** O(n)

- **Justification:** We loop through the string of characters exactly once (O(n) time). In the worst case (all open brackets), the stack will grow to the size of the string, so space is O(n).

Python

```python
# Time Complexity: O(n)
# Space Complexity: O(n)

def isValid(s):
    stack = []
    mapping = {")": "(", "}": "{", "]": "["}

    for char in s:
        if char in mapping:
            # Pop the top element if stack isn't empty, else assign a dummy
            top_element = stack.pop() if stack else '#'
            if mapping[char] != top_element:
                return False
            else:
                stack.append(char)

    return not stack

# Test Executions
print("Test A ( '()' ):", isValid("()")) # Expected: True
print("Test B ( '()[]{}' ):", isValid("()[]{}")) # Expected: True
print("Test C ( '(]' ):", isValid("(]")) # Expected: False
```

3. Test Case Executions

```
id_parentheses.py
Test A ( '()' ): True
Test B ( '()[]{}' ): True
Test C ( '(]' ): False
```

4. AI Review & Disagreement Resolution

Prompt Used: Review this solution I wrote. Do not rewrite it. 1. List any edge cases it would fail on. 2. State its time and space complexity, and explain how you arrived at each. 3. Name one thing that could be clearer.

AI Response:

1. **Edge Cases:** Handled perfectly. You guarded against popping from an empty stack by using a dummy # character, which prevents `IndexError` on inputs like `]`. Returning `not stack` correctly handles inputs like `[`, where the stack is not empty at the end.

2. **Complexity:** Time: O(n) to iterate through the string. Dictionary lookups and stack `append`/`pop` operations are O(1). Space: O(n) worst-case if all characters are opening brackets and pushed to the stack.

3. **Clarity:** It's an optimal interview solution.

Disagreement Resolution: No disagreement.

Failure Analysis & Reflection

My First Wrong Approach on the Hardest Problem (Problem 4)

On the "Valid Parentheses" problem, my first instinct was to just count the brackets. I created variables for open_round, open_square, etc., and added 1 when I saw an open bracket, and subtracted 1 for a closed bracket. I thought if they all equaled 0 at the end, it was valid. It worked for `()[]{}`. However, it completely failed on edge cases like `([)]`. I realized that simple counting destroys the *order* in which the brackets were opened. Counting only proves the correct quantities exist, not that they are nested properly.

Reflection: Where I get stuck

I realized during this homework that I primarily get stuck when a problem requires me to step away from basic loops and use a specific auxiliary data structure to manage "state." If the answer can be found by just looping and comparing values (like finding a duplicate or an anagram), I can solve it quickly. But when the problem requires tracking chronological order (like using a Stack for Parentheses or a Hash Map for Two Sum), I waste 10-15 minutes trying to force standard nested loops to work. Learning to recognize the "clues" that demand a Stack or Hash Map (like "matching pairs" or "finding complements") is the biggest habit I need to develop for interviews.